

Informe Técnico

Análisis de arquitecturas de software

NAC-PTIIGE123-00000002

COORDINACIÓN DE GESTIÓN DE DATOS Y ARQUITECTURA TECNOLÓGICA

Departamento de Planificación TI y Gestión de la Información - Datos

Dirección Nacional de Tecnología

Mayo 2023

Versión: 1.0.0

Firmas de Responsabilidad

Dirección Nacional de Tecnología		Fecha: mayo 2023	VERSIÓN 1.0.0	PÁGINAS 37
NOMBRE Y CARGO			FIRMA	
APROBADO POR:	Ing. Paulina Correa A. JEFE NACIONAL DE DEPARTAMENTO DE PLANIFICACIÓN TI Y GESTIÓN DE INFORMACIÓN- DATOS		 Firmado electrónicamente por: PAULINA ELIZABETH CORREA AGUAYO	
REVISADO POR:	Ing. Edison Cayambe Ch. COORDINADOR NACIONAL DE GESTIÓN DE DATOS Y ARQUITECTURA TECNOLÓGICA		 Firmado electrónicamente por: EDISON ESTUARDO CAYAMBE CHICAIZA	
ELBORADO POR:	Ing. Christian Arreaga D. EXPERTO NACIONAL ARQUITECTURA SOLUCIONES TECNOLÓGICAS		 Firmado electrónicamente por: CHRISTIAN GUILLERMO ARREAGA DESIDERIO	
COLABORA:	Ing. Margarita Zapata EXPERTO NACIONAL DE DESARROLLO		 Firmado electrónicamente por: ESTHER MARGARITA ZAPATA GARCIA	
COLABORA:	Ing. Cristhian Cabezas G. EXPERTO NACIONAL DE DESARROLLO		 Firmado electrónicamente por: CRISTHIAN FABIAN CABEZAS GUTIERREZ	
COLABORA:	Ing. César Paullan P. ESPECIALISTA DE SEGURIDAD INFORMÁTICA		 Firmado electrónicamente por: CESAR AUGUSTO PAULLAN PILCO	
COLABORA:	Ing. Darwin Calle L. EXPERTO NACIONAL DE CONTROL Y ASEGURAMIENTO DE CALIDAD (QA)		 Firmado electrónicamente por: DARWIN AUGUSTO CALLE LOJA	
COLABORA:	Ing. David Mayorga P. EXPERTO NACIONAL DE SEGURIDAD INFORMÁTICA		 Firmado electrónicamente por: DAVID MAYORGA POLO	
COLABORA:	Ing. Jorge Armijo EXPERTO NACIONAL DE INFRAESTRUCTURA Y OPERACIONES, CENTRO DE COMPUTO		 Firmado electrónicamente por: JORGE LUIS ARMIGO QUITO	

Tabla de contenidos

1. Introducción	4
1.1. Antecedentes.....	4
1.2. Objetivos	5
1.3. Alcance	6
2. Desarrollo	6
2.1. Argumentos.....	6
2.2. Marco Teórico.....	6
2.2.1. Industria 4.0.....	6
2.2.2. Arquitectura de microservicios	8
2.2.3. Contenedores	9
2.2.4. Kubernetes	10
2.3. Análisis de la problemática	10
2.4. Hipótesis que ha motivado la realización del trabajo	11
3. Evolución digital.....	12
3.1. Modelo de negocio	12
3.2. Proceso de transición a la nube.....	13
3.3. Cultura.....	13
3.4. El modelo de agilidad.....	13
3.5. Arquitecturas Resilientes	16
4. Metodología	17
4.1. Descripción fuentes de datos.....	17
4.2. Justificación de la información	19
5. Análisis de arquitecturas.....	19
5.1. Arquitectura tradicional.....	20
5.2. Arquitectura resiliente	23
5.3. Resultado del análisis comparativo	28
6. Conclusiones.....	31
7. Recomendaciones	32
8. Bibliografía	33
9. Glosario.....	34

Tabla de figuras

Figura 1 Revoluciones industriales	7
Figura 2 Arquitectura monolítica y de microservicios	8
Figura 3 Máquinas Virtuales y Contenedores	10
Figura 4 Pilares Evolución Digital	12
Figura 5 Modelos de Negocio	12
Figura 6 Viaje a la nube	13
Figura 7 Células End to End	15
Figura 8 Flujo de aplicaciones basadas en contenedores	16
Figura 9 Estadísticas tiempo de certificación proyectos institucionales	18
Figura 10 Componentes de la arquitectura actual	20
Figura 11 Diagrama general de despliegue arquitectura actual	22
Figura 12 Patrón Retry	24
Figura 13 Funcionamiento patrón fallback	24
Figura 14 Funcionamiento patrón Circuit Breaker	25
Figura 15 Patrón timeout	25
Figura 16 Herramientas arquitecturas resilientes	26
Figura 17 Componentes arquitectura resiliente	27

1. Introducción

1.1. Antecedentes

La interconexión a nivel mundial ha dado lugar al apareamiento de sistemas de alta demanda en línea, incluidos los sistemas gubernamentales para atención al público. Como resultado, surge la problemática común relacionada con la falta de la disponibilidad de los sistemas debido a la sobrecarga de solicitudes. Esta situación nos lleva a reflexionar sobre uno de los atributos clave de la calidad de los productos o servicios: la robustez.

En la actualidad, la baja eficiencia o indisponibilidad de los servicios generan múltiples problemas para la Institución, particularmente en lo que respecta al costo que conlleva la falta de calidad en la implementación de los servicios, las pérdidas económicas y de la imagen del Servicio de Rentas Internas. Estos problemas se traducen en inconvenientes a los usuarios internos, contribuyentes y ciudadanía en general afectando negativamente a la imagen del Servicio de Rentas Internas.

El Estatuto Orgánico de Gestión Organizacional por Procesos del Servicio de Rentas Internas, de acuerdo con su última reforma realizada mediante Resolución No. NAC-DGERCGC22-00000047 suscrito el 30 de septiembre de 2022 que rige desde el 1 de marzo de 2023, estipula entre los producto y servicios de la Dirección Nacional de Tecnología, el Entregable EOP: Tecnologías emergentes investigadas y propuestas.

La Gestión Interna del Departamento de Planificación TI y Gestión de Información-Datos siguiendo su cultura de investigación de tecnologías emergentes, identificó la necesidad de investigar la tecnología de contenerización de servicios para modernizar el despliegue de las aplicaciones bajo una arquitectura de microservicios.

La Dirección de Tecnología ha venido implementando soluciones tecnológicas basadas en los estándares institucionales los cuales están enfocados en una arquitectura monolítica.

Las arquitecturas monolíticas son un tipo de diseño de software donde todas las partes de una aplicación se integran en un único componente. En este enfoque, todas las funciones de la aplicación, como la interfaz de usuario, la lógica de negocio y el acceso a la base de datos, se encuentran dentro de un solo proceso de ejecución, es decir, contiene toda la funcionalidad requerida por el usuario y todas las conexiones entre los módulos de la aplicación, sin embargo, esta estructura presenta desafíos en el mantenimiento y la actualización. Los departamentos de Desarrollo, Seguridad Informática e Infraestructura y Operaciones se enfrentan a estas dificultades para gestionar estas tareas, la escalabilidad de las aplicaciones monolíticas es limitada ya que dependen de un mayor uso de recursos de hardware para su correcto despliegue y funcionamiento.

Mantener aplicaciones con un nivel de calidad elevado en un mundo donde las organizaciones cada día apuestan más a la automatización de sus procesos con el apoyo de la tecnología, puede convertirse en una tarea compleja y con tiempos elevados sobre todo si no se cuenta con un proceso automatizado para la integración de todas las áreas involucradas en el proceso del desarrollo de software y para el despliegue de estas, es aquí donde se contempla la opción de la adopción de una

cultura DevSecOps para la implementación de integración y despliegue continuo entre todos los ambientes que se tienen actualmente en la institución.

La implementación de DevSecOps con arquitecturas resilientes orientadas a contenedores implica la integración de los procesos de seguridad, desarrollo, pruebas, certificación y puesta en producción en una única cadena de herramientas integradas, esto permite que los Departamentos de Desarrollo, Seguridad Informática e Infraestructura y Operaciones trabajen en conjunto en un entorno colaborativo, acelerando el ciclo de vida de las aplicaciones, asegurando y mejorando la calidad de estas.

Los contenedores como pieza fundamental de construcción de implementaciones se relacionan mucho con la utilización de almacenamiento en la nube (más comúnmente en inglés llamado Cloud Computing) que es una forma de almacenar toda nuestra infraestructura en algún servidor o varios servidores ubicados en distintas partes del mundo y accesibles por la internet.

Las necesidades actuales institucionales, requieren de soluciones tecnológicas que permitan mantener un alto porcentaje de disponibilidad, escalabilidad y seguridad, para lo cual es necesario contar con herramientas que apalanquen estas necesidades. Con el uso de la tecnología de contenedores se puede implementar una mejor estrategia de recuperación ante eventos no planificados y soportar de forma más ágil, dinámica y autónoma la alta demanda de solicitudes en meses de mayor transaccionalidad.

1.2. Objetivos

Objetivo General

Mejorar los tiempos de entrega y la calidad de software, mediante el uso de tecnologías integradas para asegurar el software en todo el ciclo de vida desde la etapa de construcción hasta la puesta en producción, con lo cual se incrementa el desarrollo de la creatividad, eliminando tareas repetitivas de revisión, configuración y solución de errores comunes de servidores, para contribuir con el desarrollo de la innovación.

Mejorar la distribución de los recursos físicos/lógicos, automatizando el incremento o decremento de estos, en función de la necesidad de capacidad efectiva del uso de los recursos, mejorando la disponibilidad del servicio en las aplicaciones al automatizar la entrega y el despliegue de software utilizando ciclos de pruebas automatizadas para detectar y resolver problemas de manera temprana.

Objetivos Específicos

- ✓ Reducir el tiempo de desarrollo, pruebas, certificación hasta que el mismo se encuentra desplegado en producción.
- ✓ Mejorar la escalabilidad e implementación de las aplicaciones en tiempos de alta demanda de la transaccionalidad de forma autónoma.

- ✓ Incrementar la cantidad de despliegues en producción, atendiendo a las necesidades de mejora continua del negocio con mayor agilidad.
- ✓ Disminuir el tiempo promedio utilizado para la recuperación de un problema como resultado de un defecto de software.
- ✓ Disminuir el porcentaje de despliegues que provocan una falla en producción debido a errores por tareas manuales.

1.3. Alcance

Realizar un análisis de costo beneficio entre la arquitectura tecnológica existente en la Institución para el despliegue de servicios y una arquitectura resiliente para lograr la adopción de una plataforma de contenerización, considerando la necesidad y la capacidad institucional instalada, estableciendo comparativos de alternativas, ventajas y recomendaciones.

2. Desarrollo

2.1. Argumentos

Analizar las arquitecturas tradicionales que actualmente maneja la institución, comparándolas con las arquitecturas resilientes existentes en el mercado.

2.2. Marco Teórico

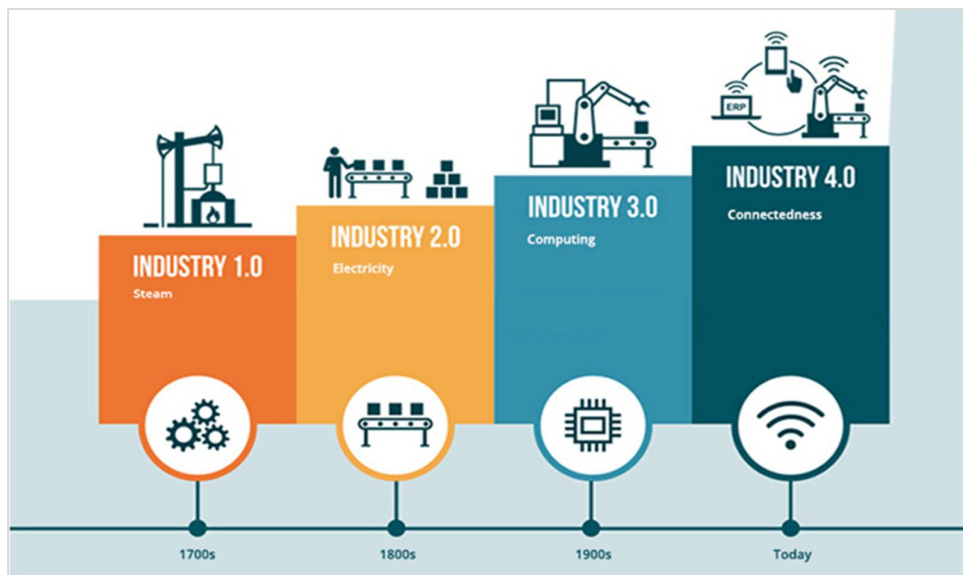
En el presente documento se realiza un análisis sobre las tecnologías asociadas para la adopción de una plataforma que permita el despliegue de contenedores.

2.2.1. Industria 4.0

Desde 1800, hemos experimentado tres revoluciones industriales. Cada una estaba impulsada por una nueva tecnología disruptiva: la mecánica del motor a vapor contribuyó a la producción en masa, un siglo después la innovación de la línea de montaje y el uso de petróleo, gas y energía eléctrica. Estas nuevas fuentes de energía, junto con comunicaciones más avanzadas a través del teléfono y el telégrafo, trajeron la producción en masa y cierto grado de automatización a los procesos de fabricación y luego se incorporaron los sistemas informáticos, las telecomunicaciones avanzadas y el análisis de datos a los procesos de fabricación.

La razón por la que se llaman "revoluciones" industriales es que la innovación que las impulsó no sólo mejoraba la productividad y la eficiencia, sino que revolucionó la forma en que se producían los bienes y cómo se ejecutaba el trabajo.

Figura 1
Revoluciones industriales



Nota: Las 4IR como apoyo para mejorar la conectividad y la productividad y algunos de los problemas sociales. Tomado de (Rajeev Gupta [mydbsyn], 2019)

Ahora estamos en la Cuarta Revolución Industrial, también conocida como Industria 4.0, que lleva al futuro la automatización y la informatización que vimos en la Tercera Revolución Industrial. La Industria 4.0 se centra en la automatización y digitalización de los procesos de producción, lo que implica una mayor eficiencia y productividad en toda la cadena de valor.

Tecnologías de la Industria 4.0

La Industria 4.0 se basa en nueve pilares tecnológicos. Estas innovaciones conectan los mundos físico y digital y habilitan sistemas inteligentes y autónomos.

- El Big Data y las analíticas de la Inteligencia Artificial
- Integración horizontal y vertical
- Computación en la nube
- Realidad aumentada (AR)
- Internet de las cosas industrial (IIoT)
- Fabricación aditiva/Impresión 3D
- Robots autónomos
- Simulación/gemelos digitales
- Ciberseguridad

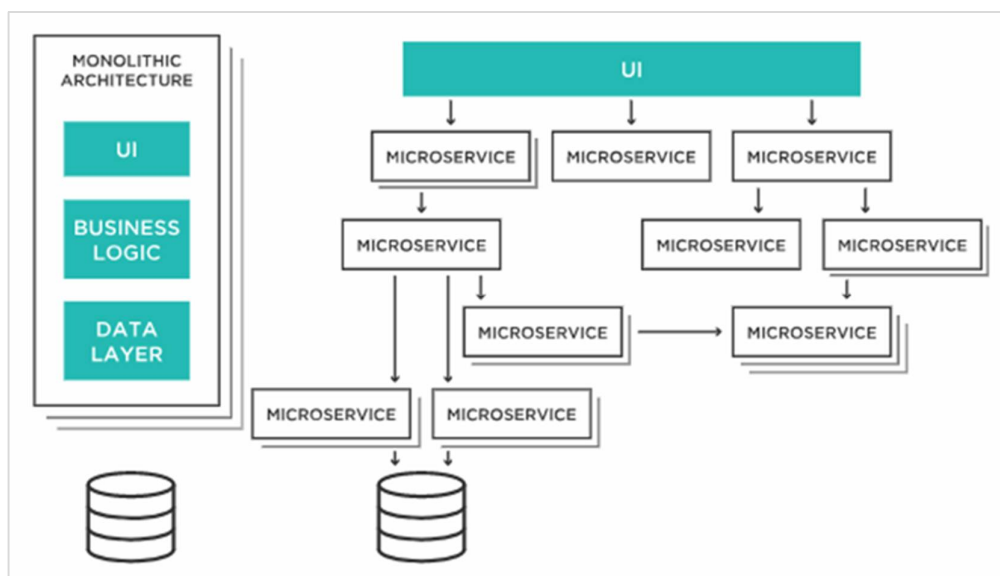
2.2.2. Arquitectura de microservicios

La arquitectura de microservicios es un enfoque de diseño de software en el que una aplicación se construye como un conjunto de servicios pequeños e independientes que se comunican entre sí mediante APIs. En este tipo de arquitectura, cada servicio realiza una función específica y puede ser desarrollado, desplegado, actualizado y escalado de forma independiente, lo que permite una mayor flexibilidad y agilidad en el desarrollo y la implementación de aplicaciones.

Cada uno de los servicios es independiente y debe implementar una funcionalidad de negocio individual dentro de un contexto delimitado. Un contexto delimitado es una división natural de una empresa y proporciona un límite explícito dentro del cual existe un modelo de dominio.

Cada servicio se comunica con otros servicios, a través de interfaces de programación de aplicaciones (APIs) estandarizadas, lo que permite que los servicios se escriban en diferentes lenguajes o en diferentes tecnologías. Esto difiere completamente de los sistemas construidos como estructuras monolíticas donde los servicios estaban inseparablemente interconectados y solo se podían escalar juntos.

Figura 2 Arquitectura monolítica y de microservicios



Nota: Muestra componentes principales de las arquitecturas monolítica y de microservicios.
Tomado de (Tibco, 2023)

Beneficios de una arquitectura de microservicios

- Los componentes de la aplicación se pueden construir en diferentes lenguajes de programación
- Se pueden mantener los flujos individuales de desarrollo e implementación continuos
- Se pueden construir aplicaciones extremadamente escalables
- Es posible el uso de opciones de implementación de función como servicios nativos de la nube
- Frecuentemente resultan en costos operativos más bajos
- El aislamiento y la estructura flexible permiten actualizaciones y mejoras compartimentadas

Cómo funciona la arquitectura de microservicios

En una arquitectura de microservicios, una aplicación compleja se desglosa en capacidades discretas que pueden desarrollarse y operar de forma independiente. Los microservicios individuales se comunican entre sí, a menudo a través de las APIs, y tienen conexiones sueltas, pero cada microservicio puede desarrollarse, implementarse y actualizarse por separado.

La implementación de microservicios suele implementarse siguiendo uno de los tres patrones siguientes:

- **Nativo de la nube.** Algunos servicios y aplicaciones establecidos de alto volumen comienzan como microservicios y permanecen en la nube. Según International Data Corporation (IDC), alrededor del 56 % de los microservicios son nativos de la nube, mientras que el 44 % restante se originaron como aplicaciones antiguas.
- **Refactorización y cambio.** Estas implementaciones comienzan en las instalaciones o en un centro de datos en el perímetro y se refactorizan para adaptarse a la arquitectura de microservicios basados en la nube. La refactorización puede incluir reasignar bases de datos y otros recursos asociados con la arquitectura monolítica, de modo que se combinen con los microservicios correspondientes.
- **Lift and shift.** Algunas organizaciones migran sus aplicaciones a una arquitectura de microservicios, sin refactorización, en una simple transición “lift and shift”.

2.2.3. Contenedores

Los contenedores son una forma de virtualización del sistema operativo. Un solo contenedor se puede usar para ejecutar cualquier componente de software, desde un micro servicio o un proceso de software a una aplicación de mayor tamaño. Dentro de un contenedor se encuentran todos los ejecutables, el código binario, las bibliotecas y los archivos de configuración necesarios. Sin embargo, en comparación con los métodos de virtualización de máquinas o servidores, los contenedores no contienen imágenes del sistema operativo. Esto los hace más ligeros y portátiles, con una sobrecarga significativamente menor. En implementaciones de aplicaciones de mayor tamaño, se pueden poner en marcha varios contenedores como uno o varios clústeres de contenedores. Estos clústeres se pueden gestionar mediante un orquestador de contenedores, como Kubernetes.

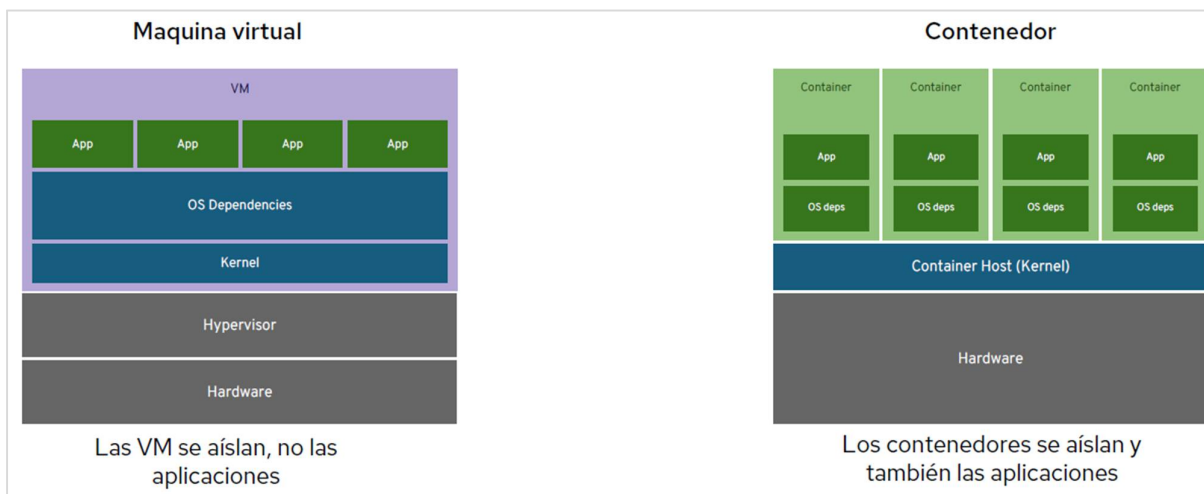
Beneficios de los contenedores

Los contenedores son una forma optimizada de crear, probar, poner en marcha y volver a poner en marcha aplicaciones en varios entornos, desde un portátil local de un desarrollador hasta un centro de datos on-premise e incluso en la nube. Algunos de los beneficios de los contenedores son:

- **Menos sobrecarga.** Los contenedores requieren menos recursos del sistema que los entornos de máquinas virtuales tradicionales o de hardware porque no incluyen imágenes del sistema operativo.
- **Mayor portabilidad.** Las aplicaciones que se ejecutan en contenedores se pueden poner en marcha fácilmente en sistemas operativos y plataformas de hardware diferentes.
- **Funcionamiento más constante.** Los equipos de DevOps saben que las aplicaciones en contenedores van a ejecutarse igual, independientemente de dónde se pongan en marcha.

- **Mayor eficiencia.** Los contenedores permiten poner en marcha, aplicar parches o escalar las aplicaciones con mayor rapidez.
- **Mejor desarrollo de aplicaciones.** Los contenedores respaldan los esfuerzos ágiles y de DevSecOps para acelerar los ciclos de desarrollo, prueba y producción.

Figura 3 Máquinas Virtuales y Contenedores



Nota: Diferencias entre máquina virtual y contenedores. Tomado de (Fedora, 2019)

2.2.4. Kubernetes

Kubernetes es una plataforma de orquestación de contenedores de código abierto que automatiza la implementación, la gestión y el escalamiento de aplicaciones.

Kubernetes permite desplegar aplicaciones fáciles de administrar e implementar en cualquier lugar y ofrece una variedad de soluciones según las necesidades de la Institución:

- Actualización de credenciales y configuración de aplicaciones sin reconstruir su imagen.
- Administración de cargas de trabajo por lotes e integración continua, reemplazando los contenedores que fallan.
- Escalabilidad de aplicaciones hacia arriba y hacia abajo en base a reglas o manualmente.
- Integración con herramientas de despliegue para DevOps.
- Transformación digital.
- Reducción de Tiempos y Esfuerzos.
- Eliminación de errores humanos.

2.3. Análisis de la problemática

Problemática en satisfacción al cliente. La cantidad de cambios necesarios para el cliente no son atendidos con rapidez debido a la complejidad que representan los procesos para llegar a las liberaciones en ambientes productivos, una vez en producción puede existir tiempos de

indisponibilidad y de respuestas elevados de los aplicativos “lentitud”, existen cambios simples que implican tiempos prolongados para su atención.

Problemática en las liberaciones. Existen tareas manuales por ambientes no homologados, las diferencias de configuración en los ambientes de pruebas y producción provocan retrasos en los pasos a producción al ser devueltos o en su defecto son ejecutados, pero puede producir errores.

Problemática en pruebas. Existen tiempos elevados en las revisiones de calidad del software, las diferencias de configuración en los ambientes de desarrollo, pruebas y producción provocan que los pasos a producción presenten errores en su ejecución debido a estas diferencias.

Problemática en desarrollo. Se busca solventar un problema o incidente, pero el tiempo de respuesta de los procesos de pruebas y liberaciones es muy largo para poder identificar si fue corregido o si se debe volver a enviar una nueva resolución, existen varios escenarios para este caso, primero si las definiciones funcionales no fueron correctas actualmente el tiempo es elevado para reaccionar ante ellas, segundo dado que los ambientes involucrados en el ciclo de desarrollo no están homologados existe probabilidad de tener productos con errores y el tiempo de reacción sigue siendo alto.

Problemática en seguridad. Los permisos de firewall deben ser otorgados por cada servidor virtual mediante su dirección IP, la asignación de permisos de firewall es proporcional al crecimiento de servidores virtuales ya sea por nuevos proyectos o mantenimientos a los aplicativos, existe una carga operativa adicional en la gestión de permisos por actualización o eliminación de los servidores. La gestión de vulnerabilidades se realiza por cada sistema operativo instalado en cada servidor virtual, lo que conlleva ejecutar un mismo proceso en varios servidores.

2.4. Hipótesis que ha motivado la realización del trabajo

El análisis de arquitecturas de software fue motivado con el objetivo de adoptar una solución tecnológica que permita gestionar y homologar el proceso del desarrollo de software, pruebas, seguridad y despliegue continuo de los servicios, mediante el uso de contenedores.

Hipótesis. La continua implementación de tecnologías y soluciones digitales modernas en la institución mejorará la eficiencia operativa, fomentará la transparencia y fortalecerá la interacción con los usuarios, contribuyentes y ciudadanía en general, lo que conducirá a una mejora en la calidad de los servicios públicos y en la confianza de la comunidad.

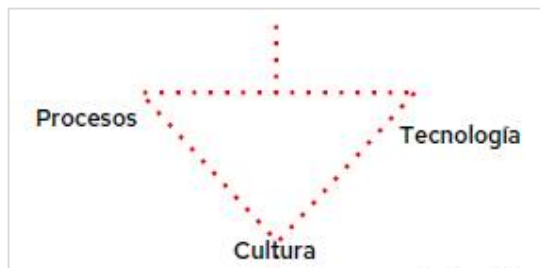
Explicación. La hipótesis plantea que la adopción de tecnologías y soluciones digitales modernas en la institución tendrá impactos positivos en varios aspectos clave para el negocio y la operación:

- Al implementar tecnologías modernas, como arquitecturas resilientes, se puede automatizar procesos, lo que conduce a una mayor eficiencia operativa. Esto permite una gestión más ágil y rápida de los procesos internos, reduciendo los tiempos de respuesta y optimizando los recursos disponibles.

- La implementación de tecnologías de contenedores en la institución puede generar ahorros significativos en el tiempo en términos de costos operativos y administrativos.

3. Evolución digital

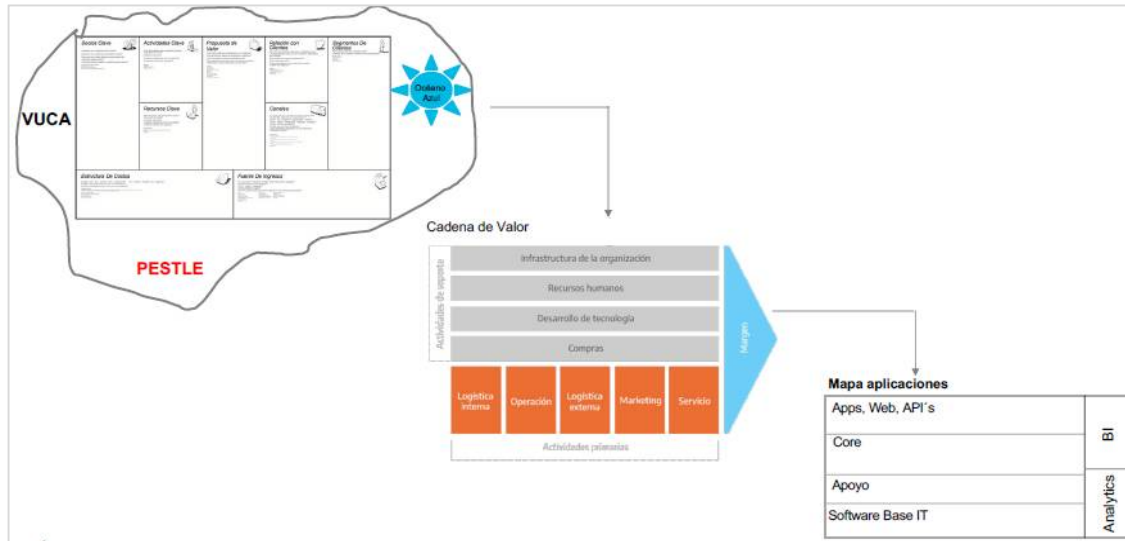
Figura 4 Pilares Evolución Digital



Nota: Tomado de (Red Hat, 2022)

3.1. Modelo de negocio

Figura 5 Modelos de Negocio



Nota: Tomado de (Red Hat, 2022)

3.2. Proceso de transición a la nube

Figura 6 Viaje a la nube



Nota: Tomado de (Red Hat, 2022)

3.3. Cultura

Con el objetivo de mejorar los servicios y los tiempos entregados por TI, es necesario considerar las tendencias de tecnología, metodología y conocimiento cuyo enfoque principal es habilitar la agilidad, seguridad, control, y despliegue de mejoras; para ello es importante tomar en cuenta cambios a nivel cultural (gente y procesos).

Los contenedores reducen los conflictos entre los equipos de desarrollo y operaciones, ya que dividen las áreas de responsabilidad. Los desarrolladores se concentran en sus aplicaciones, y el equipo de operaciones se ocupa de la infraestructura. Además, gracias a que los contenedores se basan en tecnología Open Source, es posible obtener los últimos y mejores avances tan pronto están disponibles.

La orquestación de contenedores automatiza la implementación, la gestión, la escalabilidad y la conexión de red de los contenedores. Se puede aplicar en cualquier entorno en el que ejecute contenedores y permite implementar la misma aplicación en diferentes entornos sin necesidad de volver a diseñar. Además, si se utiliza junto con arquitecturas de microservicios, resultará más sencillo organizar los servicios, incluidos el almacenamiento, la conexión de red y la seguridad.

3.4. El modelo de agilidad

Trasladar la agilidad del departamento de IT, a un concepto de agilidad Empresarial y de Negocio

Por medio de la agilidad empresarial, se logra la habilidad para reaccionar y responder de una manera innovadora y creativa a los cambios en las condiciones del mercado para generar beneficios,

mediante la rápida adaptación de sus sistemas, en ambientes turbulentos o llenos de incertidumbre, por medio de una mentalidad empresarial muy parecida a otras como el *lean startup*, cuyos propósitos están centrados en evitar malgastar el tiempo, la agilidad y el poder de maniobra.

La agilidad empresarial requiere la participación de todos durante el desarrollo de las soluciones, tanto los líderes del negocio, como las áreas de Tecnología de la Información, el Departamento Legal, Marketing, Finanzas, entre otros.

Es necesario que cada una de las áreas estén alineadas con la metodología ágil y la metodología *lean* para ofrecer prácticas acordes con las necesidades y productos a la altura de las expectativas de los usuarios. En este sentido, existen cuatro principios o “mandamientos” de la agilidad empresarial, que puedes seguir para garantizar la adaptabilidad del negocio hasta convertirla en una mentalidad colectiva:

a. Customer Centric

Considerar la agilidad empresarial como una metodología propia: tiene que poner al cliente en el medio de todas sus estrategias. Cada una de las tomas de decisiones a partir de esta filosofía deben pensarse desde el punto de vista del ciudadano y cómo aportará a su experiencia, de la misma forma buscar satisfacer las demandas del usuario de forma efectiva.

b. Flujo ágil

Con el segundo de los principios, los equipos de proyectos y mantenimiento podrán diseñar o modificar flujos de trabajo con base en la metodología centrado en el ciudadano, es decir, repensar cómo las tareas, actividades e iniciativas del negocio están agregando valor o no.

Tanto los métodos lean como la agilidad empresarial están basados en la identificación de etapas que desperdician los recursos de la empresa y aquellos que son esenciales para la satisfacción del consumidor.

A partir de ello, se pueden crear nuevos procesos y operaciones que cumplan un propósito claro en la generación de valor y facilitar la entrega de productos y servicios de calidad.

c. Disposición a innovar

La incorporación de procesos de mejora continua, bajo el contexto empresarial actual, es complejo sin ponerlo en contexto de la cultura organizacional. En este sentido, la creatividad como activo de negocios es parte esencial del proceso de agilidad empresarial, comprendiendo que es la base fundamental de la innovación.

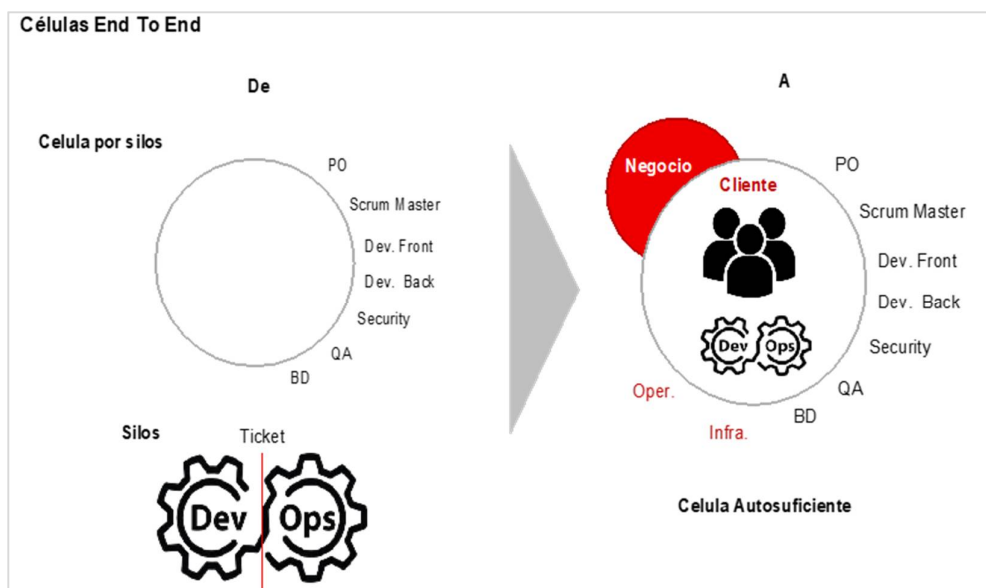
La innovación no es una responsabilidad exclusiva de los líderes de la organización, ya que la agilidad empresarial proporciona una perspectiva diferente, agregando a cada uno de los colaboradores de todas las áreas del SRI como piezas clave de este principio. Esto significa que la capacitación y la autonomía son valores que la institución debe promover y proteger con el objetivo de crear metodologías y procesos de trabajo adecuados para cumplir con los cambios del mercado.

d. El modelo de DevOps

Son cambios significativos que deben incluir el paso de un enfoque de agilidad a un enfoque de DevOps:

- i. DevOps automatizado: Se requiere contar con la creación de pipelines para soportar la gestión de CI/CD, con la mínima intervención humana.
- ii. DevOps con células autosuficientes y equipos de Team Topologies (IT): Los equipos de ingeniería deben gestionar sus actividades con mayor velocidad para ofrecer valor a los usuarios. Para seguir el ritmo de la demanda, es necesario adoptar prácticas ágiles y prácticas de DevOps, que permitan un despliegue en plazos más cortos, faciliten la frecuencia de implementación, mejoren la cultura del equipo y aumenten la colaboración entre los equipos y departamentos.

Figura 7 Células End to End

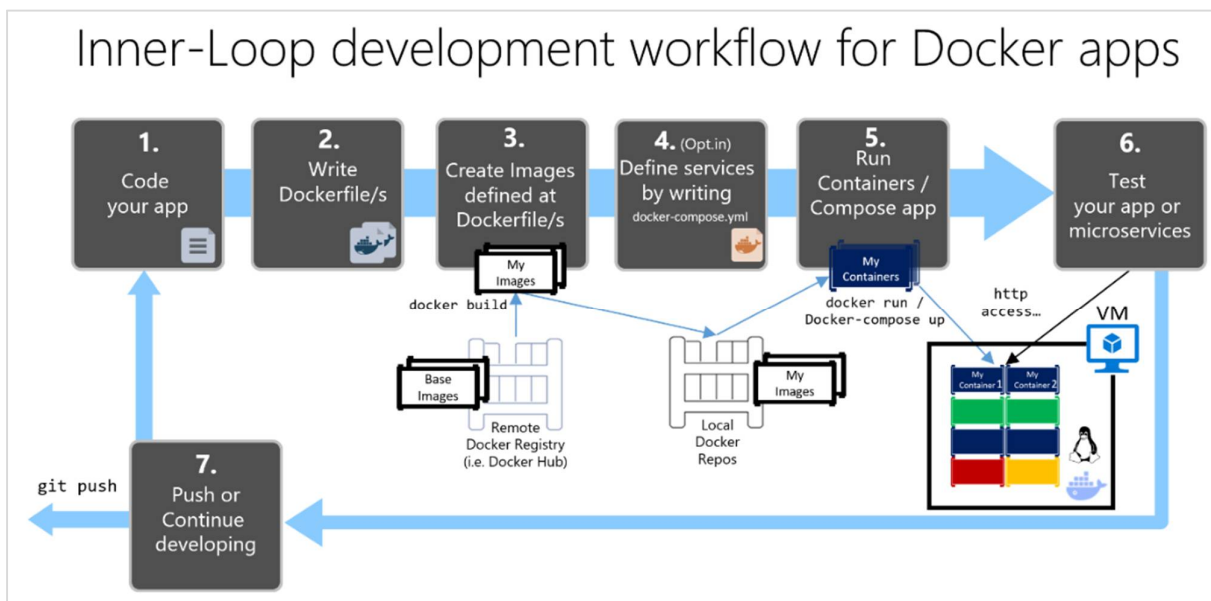


Nota: Tomado de (Red Hat, 2022)

- iii. **Devops que incluye DevSecOps en la etapa de desarrollo de aplicaciones:** El diseño de aplicaciones considerando las buenas prácticas de seguridad desde el inicio del desarrollo de toda nueva aplicación.
- iv. **Devops con métricas**
 - Lead Time for Change:** Tiempo transcurrido desde que hacemos un *commit* de código fuente hasta que el mismo se encuentra en producción.
 - Deployment Frequency:** La frecuencia con la que hacemos despliegues en producción.
 - Mean Time To Restore:** El tiempo promedio utilizado para recuperarnos de un problema como resultado de un despliegue de software.
 - Change Failure Rate:** El porcentaje de despliegues que provocan una falla en producción.

Las dos primeras métricas están relacionadas con la velocidad de cambio. Y como la innovación se obtiene experimentando a alta velocidad, se admite que inevitablemente vamos a producir errores. DevOps propone “fallar rápido”, y para esto es importante que monitoreemos las últimas dos métricas, las cuales miden cuántas fallas introducimos y cuán rápido podemos remediarlas. Mejorar la agilidad de negocio implica aumentar el *Lead Time for Change* y *Deployment Frequency*, junto con reducir el *Mean Time To Restore* y *Change Failure Rate*.

Figura 8 Flujo de aplicaciones basadas en contenedores



Nota: Tomada de (Microsoft, 2023)

3.5. Arquitecturas Resilientes

En la actualidad se observa cómo las empresas están destacándose por arquitecturas de microservicios, ya que les otorga muchos beneficios como son:

- **Desacoplamiento:** Los contenedores reducen los conflictos entre los equipos de desarrollo y operaciones, ya que dividen las áreas de responsabilidad. Los desarrolladores se concentran en sus aplicaciones, y el equipo de operaciones se ocupa de la infraestructura
- **Aislamiento de fallos:** Los contenedores ofrecen a las aplicaciones basadas en microservicios una unidad para la implementación de aplicaciones y un entorno de ejecución autónomo ideales. A su vez, permiten ejecutar en microservicios múltiples partes de una aplicación de forma independiente, en el mismo hardware, con mucho más control sobre las partes y los ciclos de vida individuales.
- **Facilidad en mantenimiento:** Los contenedores ayudan a acelerar los ciclos de desarrollo y las operaciones de TI. Las aplicaciones en contenedores se pueden implementar en diferentes infraestructuras sin aplicar cambios. Los microservicios en contenedores se pueden reutilizar en

otros proyectos. Además, los contenedores admiten las metodologías de DevOps y de integración y distribución continuas (CI/CD) que unifican las operaciones y el desarrollo y aceleran los procesos.

- **Agilidad (inclusión de DevOps):** La organización en contenedores automatiza la implementación, la gestión, la escalabilidad y la conexión en red de los contenedores.
- **Escalabilidad:** Las tecnologías de nube híbrida y contenedores le permiten expandir las aplicaciones, los servicios y los recursos en todos los entornos in situ y de nube pública, privada e híbrida de forma independiente y rápida. Esto garantiza la confiabilidad y el rendimiento en todo momento.
- **Portabilidad:** La organización de los contenedores puede aplicarse en cualquier entorno en el que se ejecuten estos elementos, incluidos los servidores locales y los entornos de nube pública y privada.
- **Seguridad:** La seguridad de los contenedores debe estar integrada y ser permanente, además de respaldar la estrategia de seguridad general de la empresa.

4. Metodología

Para generar la comparación entre la arquitectura actual y una arquitectura resiliente, se utilizan datos estimados de tiempos del proceso de Gestión Tecnológica que involucra a los procesos institucionales: Diseño del Servicio, Implementación y Liberación.

4.1. Descripción fuentes de datos

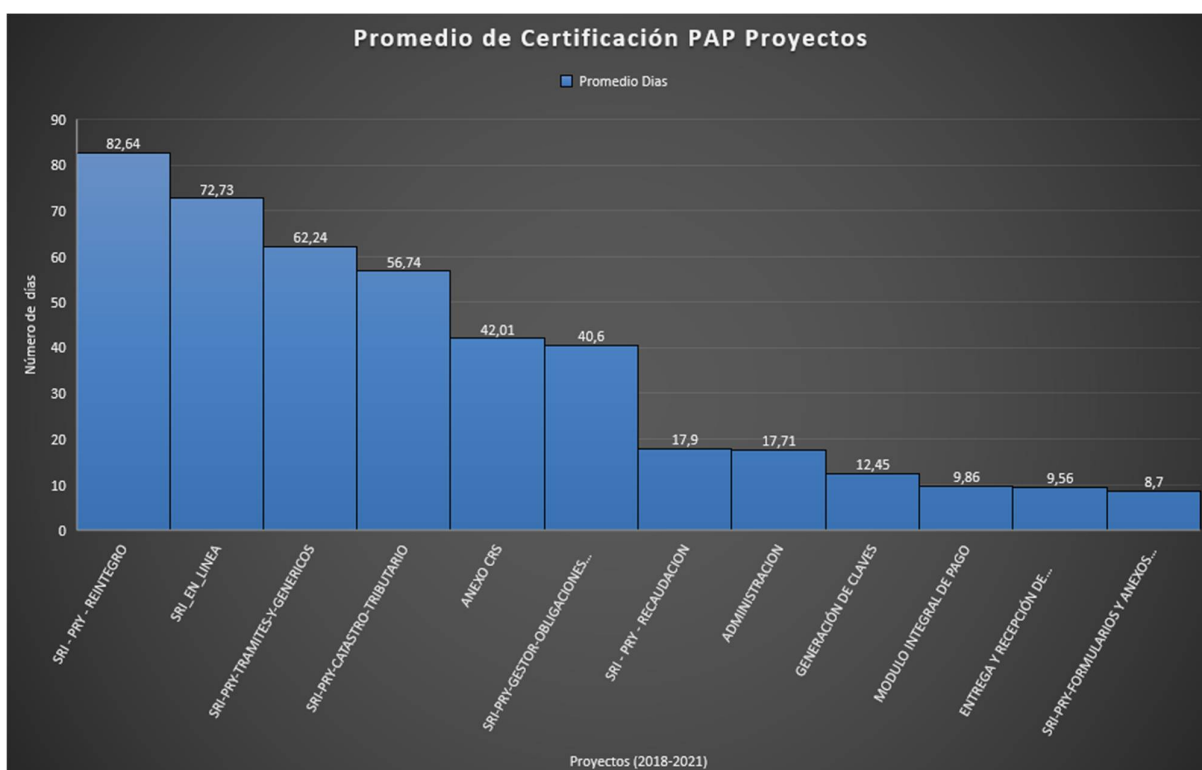
Los datos se han obtenido aplicando el juicio de expertos con uso de metodologías ágiles:

Tiempos obtenidos en la ejecución del Proceso de Gestión Tecnológica			
Procesos involucrados	Actividad	Tiempo Proyectos	Tiempo Mantenimiento
Diseño del Servicio Tecnológico	Capacidad	10 días	5 días
	Disponibilidad	7 días	3 días
Implementación de Servicios Tecnológicos	Construcción de Aplicativos Informáticos	12 meses	3 semanas
	Pruebas	6 meses	4 días
	Liberaciones	1 semana	1 hora
Operación de Servicios Tecnológicos	Gestión de Infraestructura S.O.	1 semana	1 semana
	Gestión de Infraestructura App	2 semanas	1 semana
	Despliegues por instancia	1 hora	1 hora

Tiempos obtenidos en la ejecución del Proceso de Gestión Tecnológica			
	Monitoreo de Infraestructura, redes y seguridades	Estabilización: 2 semanas Operación: Constante	Constante

A continuación, se detalla las estadísticas de los últimos proyectos mediante análisis de datos históricos registrados en el Aplicativo de Gestión TI de los pasos a producción de los proyectos más representativos:

Figura 9
Estadísticas tiempo de certificación proyectos institucionales



El promedio del tiempo desde la entrega del desarrollo para la ejecución de pruebas y hasta el despliegue en los ambientes de producción, es elevado tal como se muestra en la figura anterior; por lo tanto, se espera que con la adopción de una plataforma de contenedores se optimicen los tiempos.

4.2. Justificación de la información

La información obtenida nos muestra el tiempo elevado que toma la ejecución de las diferentes actividades en los procesos relacionados al ciclo de vida del desarrollo de aplicaciones, con el uso de plataformas basado en contenedores se pretende reducir el tiempo mediante la automatización de ejecución de tareas manuales.

Según la experiencia de varios clientes de plataformas de contenerización indican que el proceso de desarrollo ha cambiado ya que cuentan con más tiempo para el diseño, desarrollo y pruebas, debido a que los últimos pasos de la implementación son más eficientes y automatizados logrando ejecuciones más cortas. Se incrementa la calidad al hacer más pruebas en menos tiempo y se logra implementar más frecuentemente y realizando mayor cantidad de ensayo-error en nuevas funcionalidades para el cliente.

En el siguiente cuadro se cuantifica estas mejoras de tiempo:

Volumen de desarrollo por año	Antes de Contenedores	Después de Contenedores	Diferencia	% Beneficio
Nuevas aplicaciones	9.3	13.4	4.1	44%
Nuevas características	245	723	479	196%

Nota: Tomada de (IDC Business Value, 2021)

5. Análisis de arquitecturas

De acuerdo con la Norma ISO/IEC 42010:2007 se define a la arquitectura como “La organización fundamental de un sistema, expresada en sus componentes, sus relaciones entre sí y con el ambiente, y los principios que gobiernan su diseño y evolución”.

De acuerdo al Software Engineering Institute (SEI), la Arquitectura de Software se refiere a “las estructuras de un sistema, compuestas de elementos con propiedades visibles de forma externa y las relaciones que existen entre ellos.”

A continuación, se muestra un comparativo entre la arquitectura de software actual o tradicional y la nueva arquitectura propuesta o arquitectura con características de resiliencia.

Cabe mencionar que para dicho análisis comparativo se ha tomado en cuenta los componentes que conforman exclusivamente la construcción de aplicaciones y no aquellos que conforman otras capas orientadas a la infraestructura dado que estas no se ven impactadas por el alcance del cambio propuesto en este documento.

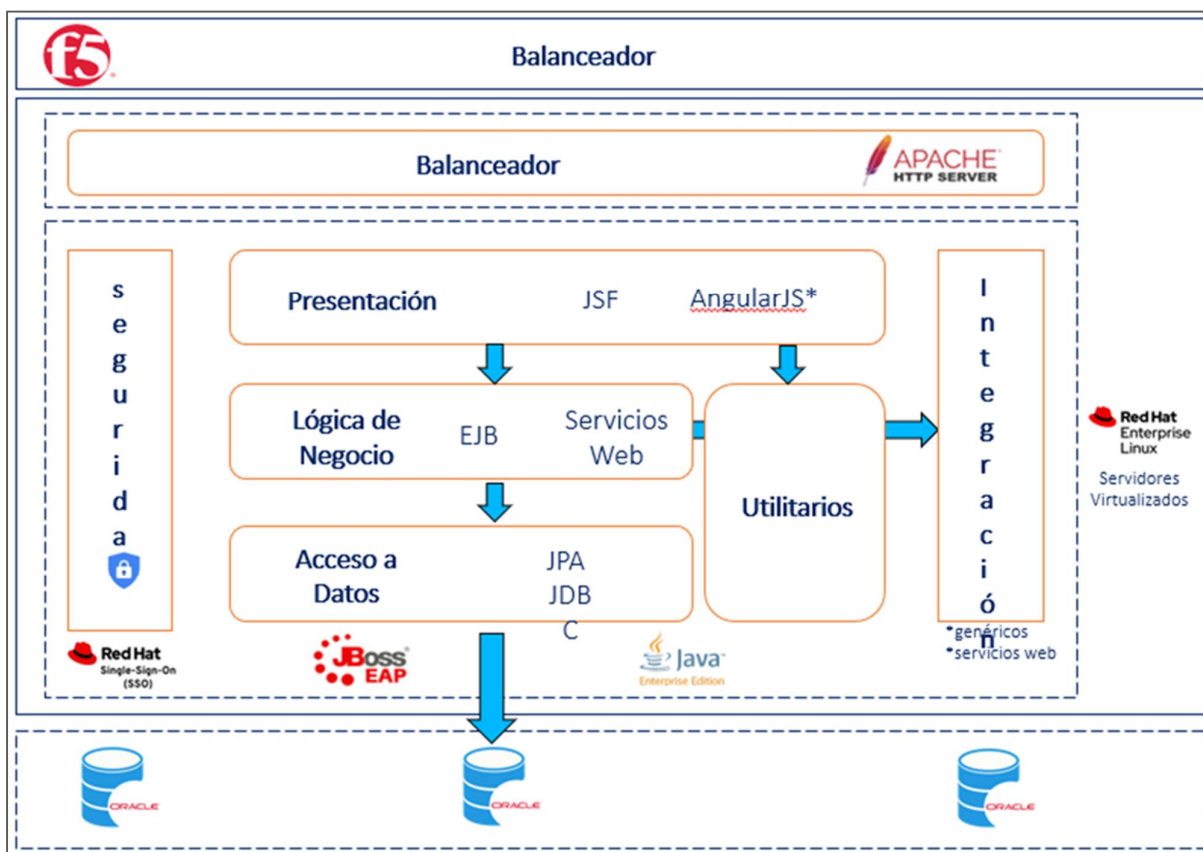
5.1. Arquitectura tradicional

La arquitectura actual utilizada para el desarrollo de aplicaciones obedece principalmente al tipo de aplicaciones que se construyen basadas en tecnología Java Enterprise Edition (JEE) y por ende desplegados en un JEE Container o servidor de aplicaciones que se encuentra desplegado en una máquina virtual cuyo sistema operativo es Red Hat Enterprise Linux.

Estas aplicaciones hacen uso del patrón MVC el mismo que es un patrón de arquitectura de software que separa los datos y la lógica de negocio de una aplicación de la interfaz de usuario y el módulo encargado de gestionar los eventos y las comunicaciones. Para ello MVC propone la construcción de tres componentes distintos que son el modelo, la vista y el controlador, es decir, por un lado, define componentes para la representación de la información, y por otro lado para la interacción del usuario. Los componentes MVC junto a otros componentes transversales tales como la base datos, balanceadores de diferentes capas y mecanismos de integración conforman la arquitectura actual tecnológica de la institución.

A continuación, se muestra un gráfico a alto nivel de estos componentes, sus tecnologías usadas y las relaciones entre ellos.

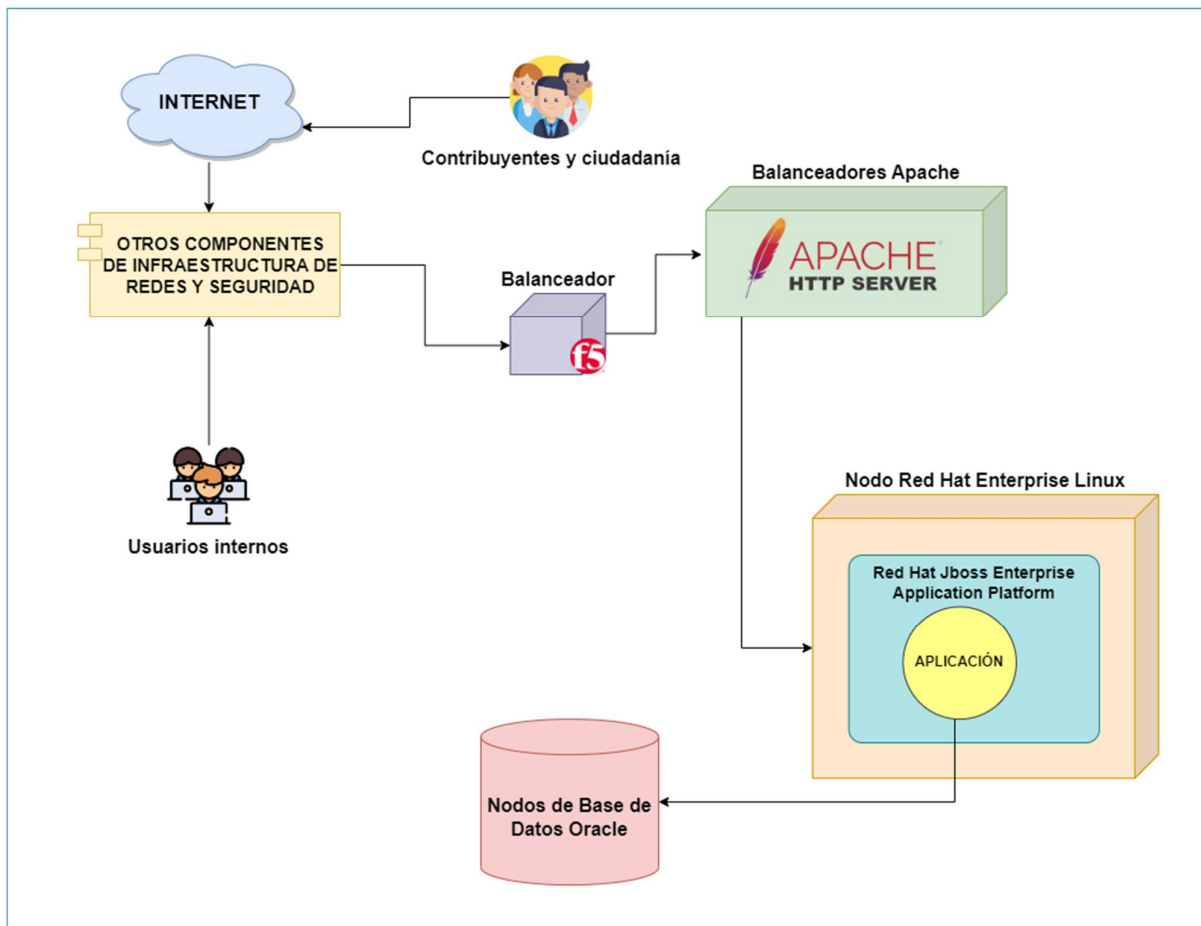
Figura 10 Componentes de la arquitectura actual



Componente	Descripción
Balancedor	Balanceador de carga de capa 4 y 7
Balanceador Apache	Capa de balanceadores por software (Apache) que redistribuye el tráfico hacia los servidores de aplicación.
Vista (Presentación)	Esta capa o componente permite presentar de manera visual la información a usuarios externos o usuarios internos. Las tecnologías utilizadas son JSF y Angular.
Lógica de Negocio (Controlador)	Esta capa o componente actúa como intermediario entre el Modelo (Acceso a datos) y la Vista (Presentación), gestionando el flujo de información entre ellos y las transformaciones para adaptar los datos a las necesidades de cada uno. Las tecnologías utilizadas en esta capa son EJB, JAX-WS, JAX-RS.
Acceso a datos (Modelo)	En esta capa se encuentra el modelo que tendrá siempre una representación de los datos del dominio, es decir, aquellas entidades que nos servirán para almacenar información del sistema que estamos desarrollando. Las tecnologías utilizadas en esta capa son JPA y JDBC.
Utilitarios	Este componente representa el conjunto de utilitarios que encierran funcionalidades utilizadas por los demás componentes como por ejemplo funcionalidades de validaciones comunes o transformaciones. La tecnología utilizada en esta capa es JSE.
Seguridad	Esta capa contiene los componentes de seguridad para el control de la autenticación, autorización y en general el acceso a las aplicaciones. El componente utilizado para la autenticación es Red Hat Single Sign On y otros componentes desarrollados bajo el estándar JEE.
Integración	En esta capa se encuentran representados los mecanismos de integración que se usan para que las aplicaciones puedan interactuar entre sí. Estos mecanismos se encuentran implementados utilizando el estándar JEE.

Adicionalmente se muestra un esquema general de despliegue basado en la arquitectura actual:

Figura 11
Diagrama general de despliegue arquitectura actual



Con el uso actual de la arquitectura vigente, se ha evidenciado a lo largo del tiempo algunas dificultades entre ellas:

- Alto acoplamiento “dependencias”:** Existen aplicaciones que por dependencias transitivas llegan a requerir hasta 9 artefactos para su despliegue y no necesariamente son usadas dentro de la lógica del giro de negocio de la aplicación.
- Escalamiento ineficiente:** Si un artefacto de una aplicación requiere mayor capacidad, la arquitectura actual obliga a escalar la instancia completa de la aplicación.
- Tiempo de indisponibilidad “recuperación”:** Cuando se requiere dar mantenimiento a un artefacto de una aplicación, se genera en toda la aplicación un tiempo de inactividad.
- Fallos totales:** Cuando un artefacto de una aplicación falla, degrada por completo el funcionamiento de la aplicación y/o de las aplicaciones que conviven en las mismas instancias.
- Mantenimiento complejo entre aplicaciones:** Existen artefactos transversales que cuando son actualizados, es necesario que todas las aplicaciones sean actualizadas, lo cual genera un mantenimiento complejo entre estas.

- f) **Crecimiento de la infraestructura:** Si una aplicación con sus dependencias requiere escalamiento horizontal se ejecutan tareas manuales hasta el límite de Hardware permitido.

5.2. Arquitectura resiliente

La resiliencia corresponde a un cambio de paradigma, Christophe Bobin, responsable de negocios en Axians Communication & Systems Paris (una empresa de la marca TIC de VINCI Energies) y especialista del sector industrial sintetiza que *“La resiliencia informática es la capacidad de garantizar la disponibilidad del servicio que se presta a una empresa”*. (theagilityeffect, 2022)

Un sistema de información debe poder seguir funcionando en caso de presentar un incidente (avería, siniestro, pico de actividad, ciberataque, etc.). Y es fundamental que pueda volver a su estado habitual de funcionamiento con la mayor rapidez posible y el menor número de impactos negativos.

Así pues, la resiliencia informática limita el tiempo que dura la interrupción de la actividad y las pérdidas de explotación asociadas, a la vez que garantiza una rápida restauración de los datos. Como se puede inferir la resiliencia en el software depende de algunos aspectos como la redundancia de la infraestructura y de los datos, la ciberseguridad, el desarrollo y despliegue de las aplicaciones, entre otras.

Como parte de la planificación de renovación tecnológica se ha planteado como objetivo institucional la adopción de este nuevo paradigma, a continuación, podemos revisar algunos aspectos de la arquitectura resiliente propuesta enfocada en la ejecución y despliegue de las aplicaciones.

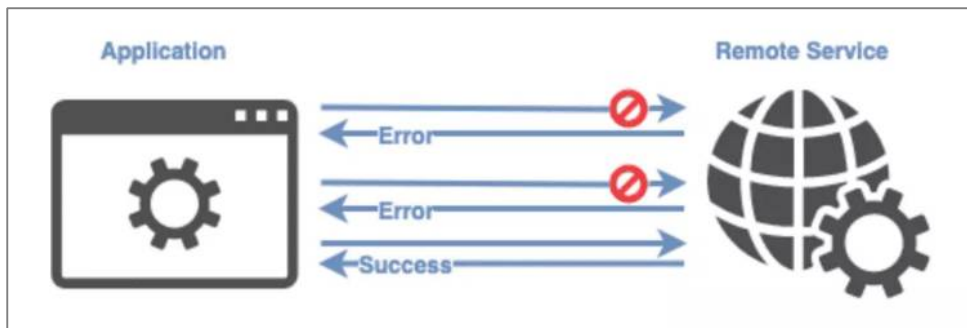
Para la aplicación de una arquitectura resiliente es necesario contar con una plataforma de contenedores que nos permita implementar los principales patrones de arquitectura orientados a este fin como los son:

- Reintento (Retry)

En aplicaciones distribuidas, donde se realizan constantes comunicaciones entre servicios y/o recursos externos, se pueden producir errores temporales o transitorios (transient failures) cuando se interactúa con estos entornos. Estos fallos pueden estar causados por diferentes motivos, entre los más comunes encontramos las pérdidas momentáneas de conexión a la red, servicios temporalmente no disponibles, tiempos de respuesta excedidos, etc.

Normalmente estos errores se solucionan de manera automática y en un breve lapso de manera que si el servicio o recurso vuelve a ser invocado inmediatamente responde de manera correcta. Un ejemplo clásico de error transitorio es el fallo de conexión con la base de datos debido a un pico de conexiones simultáneas que exceden el máximo número permitido por configuración.

Figura 12 Patrón Retry



Nota: Tomado de (Apiumhub, 2019)

- Fallback

Este patrón permite que su servicio continúe con la ejecución en caso de una solicitud fallida a otro servicio. En lugar de abortar la ejecución se da una ruta de código alternativa. Este patrón se utiliza cuando falla la ruta del código original y proporciona un mecanismo que permitirá que el cliente del servicio responda un valor a través de medios alternativos. Los valores alternativos no siempre son posibles, pero pueden aumentar en gran medida su resiliencia general si se usan con cuidado.

Figura 13 Funcionamiento patrón fallback

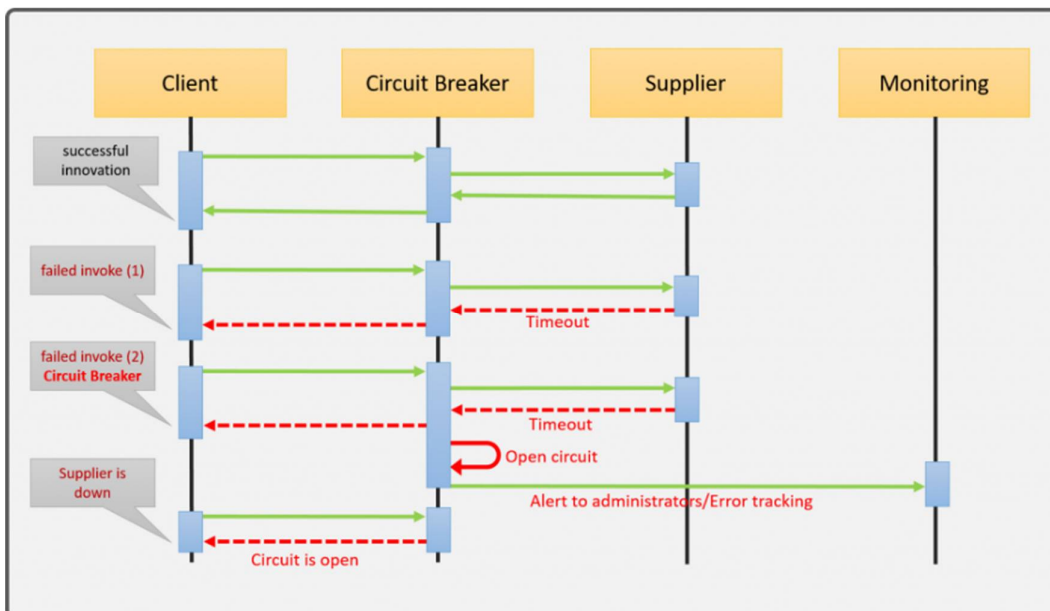


Nota: Tomado de (Faye, 2022)

- Circuit Breaker

El patrón Circuit Breaker busca evitar un fallo en cascada cuando una petición a un servicio da error, es muy parecido a un fusible, el cual se funde para evitar que una descarga eléctrica afecte al circuito, con la única diferencia de que, en software, este fusible se puede restaurar cuando el problema se ha solucionado, es decir manejando un estado en la comunicación entre los servicios, de forma que se pueda abrir o cerrar en función del comportamiento del servicio destino.

Figura 14 Funcionamiento patrón Circuit Breaker

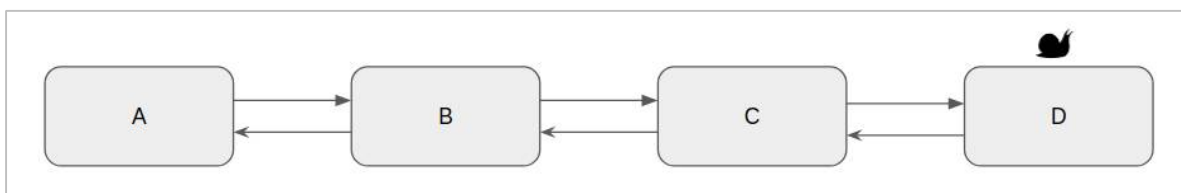


Nota: Tomado de (Blancarte, 2018)

- Timeout

El objetivo de este patrón es evitar esperas 'infinitas' en las peticiones a los servicios en situaciones de fallo, pero sin una respuesta de fallo, cuando hay varios servicios (A, B, C y D), un servicio (A) puede depender del otro servicio (B), que a su vez puede depender de C y así sucesivamente. A veces, debido a algún problema de red, es posible que el Servicio D no responda como se esperaba. Esta lentitud podría afectar los servicios posteriores, hasta el Servicio A y bloquear los subprocesos en los servicios individuales.

Figura 15 Patrón timeout

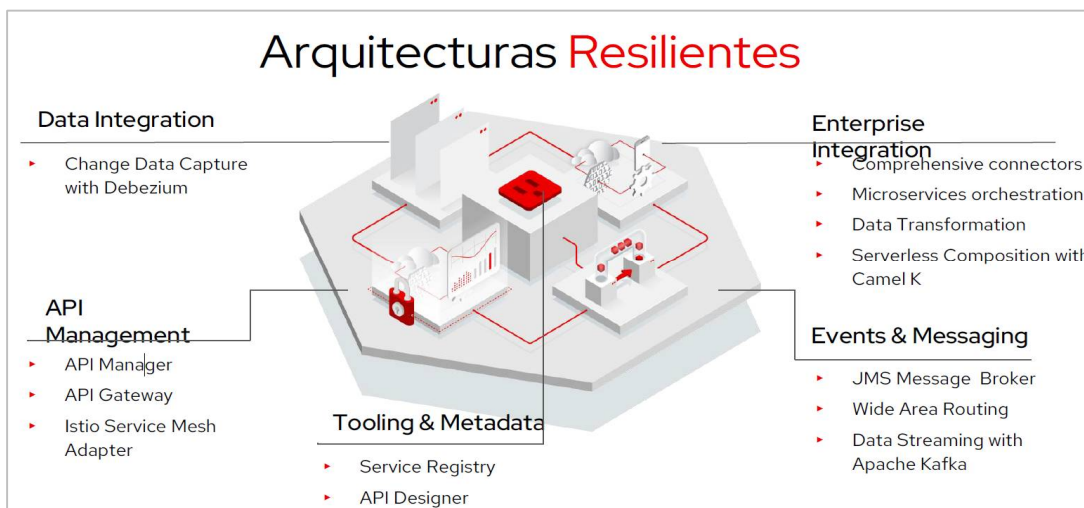


Nota: Tomado de (Vinsguru, 2020)

Cuando se encuentra con latencia en la comunicación de su servicio y no está seguro de cuál es la causa principal, el enfoque preferido es simplemente no esperar la respuesta. Es fundamental implementar una estrategia de tiempo de espera entre su servicio y la comunicación de servicio a través de la red.

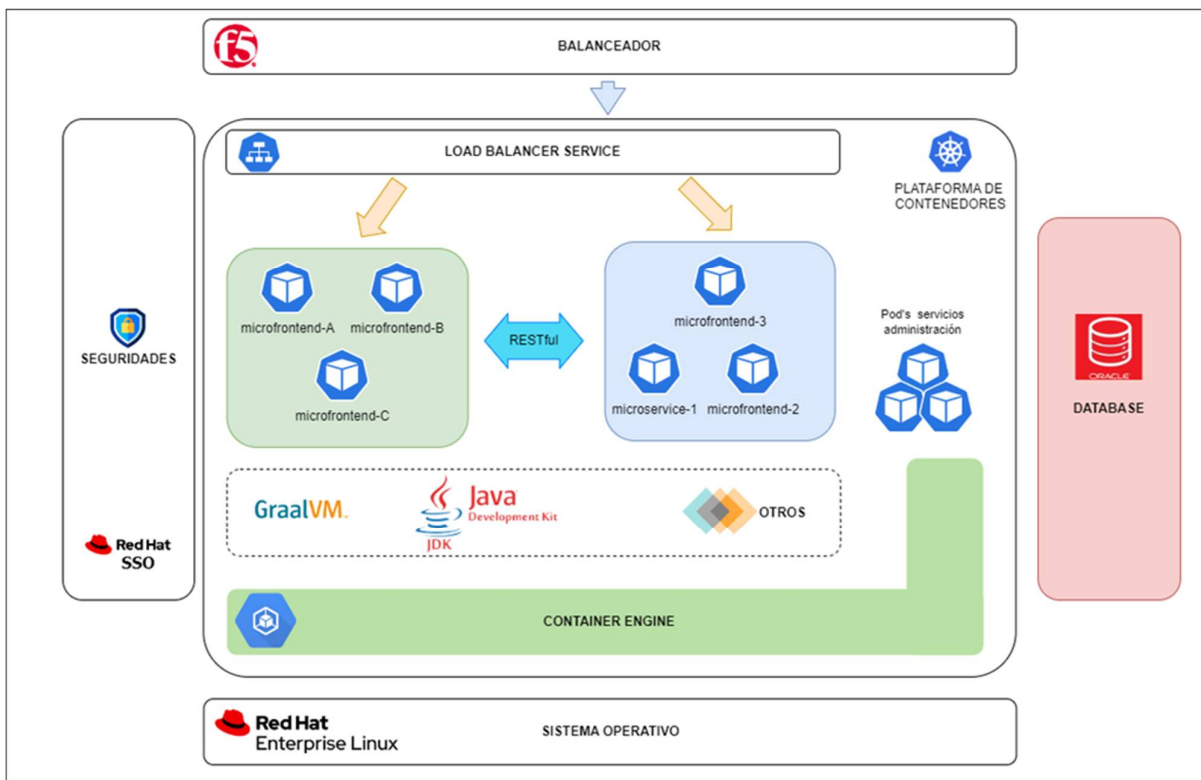
Con la finalidad de apoyar a la resiliencia existen un conjunto de patrones y herramientas adicionales tales como aquellos para la administración de los servicios, el manejo de la data, orientados al manejo de eventos entre otros que pueden desplegarse en las plataformas de contenedores como las que se muestran a continuación.

Figura 16
Herramientas arquitecturas resilientes



La arquitectura propuesta para el despliegue de aplicaciones resilientes se puede observar en el siguiente diagrama:

Figura 17 Componentes arquitectura resiliente



Como se puede observar la arquitectura anterior se centra en el despliegue de aplicaciones basado en contenedores, de esta manera los componentes ya existentes: base datos, capa de seguridad y balanceador de capa 4, no se ven afectados.

En la plataforma de contenedores se desplegarán las aplicaciones basadas en nuevas tecnologías de desarrollo que en conjunto con los servicios que vienen incluidos soportará un esquema de funcionamiento resiliente.

Esta arquitectura tiene como objetivo proveer los siguientes beneficios:

- a) Aplicaciones modulares con componentes independientes
- b) Escalamiento eficiente
- c) Mantenimientos con bajo tiempo de indisponibilidad
- d) Aislamiento de fallos
- e) Bajo acoplamiento entre aplicaciones

5.3. Resultado del análisis comparativo

Arquitectura monolítica o tradicional

Ventajas

- **Implementación sencilla:** un único archivo o directorio ejecutable facilita la implementación.
- **Desarrollo:** desarrollar una aplicación compilada con una única base de código es más sencillo.
- **Rendimiento:** en una base de código y un repositorio centralizados, una API suele realizar la misma función que muchas API en el caso de los microservicios.
- **Pruebas simplificadas:** una aplicación monolítica es una unidad única y centralizada, por lo que las pruebas integrales se pueden hacer más rápido que con una aplicación distribuida.
- **Depuración sencilla:** con todo el código ubicado en un solo lugar, es más fácil rastrear las solicitudes y localizar incidencias.

Desventajas

- **Velocidad de desarrollo más lenta:** con una aplicación grande y monolítica, el desarrollo es más complejo y lento.
- **Escalabilidad:** no se pueden escalar componentes individuales.
- **Fiabilidad:** si hay un error en algún módulo, puede afectar a la disponibilidad de toda la aplicación.
- **Barrera para la adopción de tecnología:** cualquier cambio en el marco o el lenguaje afecta a toda la aplicación, lo que hace que los cambios suelen ser costosos y lentos.
- **Falta de flexibilidad:** un monolito está limitado por las tecnologías que se utilizan en él.
- **Implementación:** un pequeño cambio en una aplicación monolítica requiere una nueva implementación de todo el monolito.

Arquitectura resiliente

Ventajas

- **Agilidad:** promueve formas ágiles de trabajar con equipos pequeños que implementan con frecuencia.
- **Escalado flexible:** si un microservicio está alcanzando su capacidad de carga, se pueden implementar rápidamente nuevas instancias de ese servicio al clúster.
- **Implementación continua:** Se logran ciclos de lanzamiento frecuentes y más rápidos.
- **Muy fácil de mantener y probar:** los equipos pueden hacer pruebas con el código y dar marcha atrás si algo no funciona como esperan. Esto facilita la actualización del código y acelera el tiempo de salida a producción de nuevas funcionalidades. Además, es fácil aislar y corregir fallos y errores en los servicios individuales.
- **Implementación independiente:** las aplicaciones se despliegan como unidades individuales, por lo que permiten una implementación independiente rápida y sencilla de funciones individuales.

- **Flexibilidad tecnológica:** con esta arquitectura, los equipos pueden elegir con libertad las herramientas que desean.
- **Alta fiabilidad:** Se pueden implementar cambios para un servicio en concreto sin el riesgo de que se caiga toda la aplicación.
- **Equipos más satisfechos:** los equipos que trabajan con arquitecturas resilientes están mucho más satisfechos, ya que son más autónomos y pueden compilar e implementar de forma independiente, sin esperar semanas a que se apruebe una solicitud de incorporación de cambios.

Desventajas

- **Desarrollo descontrolado:** los microservicios suman complejidad en comparación con las arquitecturas monolíticas, ya que hay más servicios en más lugares creados por varios equipos. Si el desarrollo descontrolado no se gestiona adecuadamente, se reduce la velocidad de desarrollo y el rendimiento operativo.
- **Costes exponenciales de infraestructura:** cada nuevo microservicio puede tener su propio coste para el conjunto de pruebas, los manuales de estrategias de desarrollo, la infraestructura de alojamiento, las herramientas de supervisión, etc.
- **Más sobrecarga organizativa:** los equipos deben agregar otro nivel de comunicación y colaboración para coordinar las actualizaciones e interfaces.
- **Desafíos para la depuración:** cada aplicación tiene su propio conjunto de registros, lo que complica la depuración. Además, un solo proceso empresarial puede ejecutarse en varias máquinas, lo que aún lo dificulta más.
- **Falta de estandarización:** sin una plataforma común, puede haber una proliferación de idiomas, de estándares de registro y de supervisión.
- **La propiedad no está clara:** a medida que se introducen más servicios, también lo hace el número de equipos que los ejecutan. Con el tiempo, se hace difícil conocer los servicios disponibles que un equipo puede aprovechar y con quién hay que hablar para obtener asistencia.

Característica	Arquitectura tradicional monolítica	o Arquitectura resiliente
Implementación sencilla	2	1
Facilidad de Depuración	3	1
Agilidad en el desarrollo	1	3
Control en el desarrollo	3	2
Fiabilidad	1	3
Flexibilidad en la adopción tecnológica	1	3
Escalamiento flexible	1	3
Implementación independiente	1	3
Satisfacción de los equipos	1	3
Sobrecarga organizativa	1	2
Facilidad de mantenimiento	1	3

Facilita los procesos de CI/CD	1	3
Total	17	30

Nota: Para la ponderación aplicada en la tabla anterior tomar en cuenta los siguientes valores donde 3 es la calificación más alta.

1 = Bajo

2 = Medio

3 = Alto

6. Conclusiones

- Para la adopción de una arquitectura resiliente es importante implementar una solución de gestión de contenedores que garantice la continuidad, disponibilidad, escalabilidad y seguridad de los servicios desplegados en contenedores para las aplicaciones del Servicio de Rentas Internas, con el objetivo de brindar un mejor servicio a los usuarios, contribuyentes y a la ciudadanía en general, logrando alcanzar los siguientes beneficios:
 - o **Beneficios en seguridad.** Se reduce la creación de reglas de permisos de firewall dado que existen menos servidores virtuales. La gestión de vulnerabilidades se simplifica al ejecutarla en una sola imagen que posteriormente será distribuida en varios contenedores.
 - o **Beneficios en pruebas.** Se reduce el tiempo de identificación de errores en la ejecución de las pruebas debido a que los ambientes se encontrarán homologados. La gestión de ambientes de pruebas y la automatización de pruebas se optimiza al reducir el tiempo de configuración de la infraestructura dado que la gestión de contenedores se torna más ágil y eficiente.
 - o **Beneficios en desarrollo.** Mejora la calidad de software, así como el rendimiento. También permite el despliegue de software con mayor eficiencia. Garantiza la portabilidad, escalabilidad y seguridad.
 - o **Beneficios en operaciones.** Facilita mantener la infraestructura homogénea ya que puede soportar diferentes versiones de aplicaciones. Facilita el monitoreo de los recursos asignados. Mayor compatibilidad entre aplicaciones y sistemas operativos. Facilidad para escalar aplicaciones de manera eficiente y mayor velocidad para desplegar aplicaciones en diferentes ambientes.
- La adopción de una plataforma de contenedores permitirá obtener elasticidad para modificar rápidamente la disponibilidad en respuesta a picos o estabilizaciones de la demanda.
- La implementación de una plataforma de contenedores permite agilizar el despliegue de las aplicaciones manteniendo la misma gestión en los procesos de base de datos, de seguridad y de balanceador de capa 4.
- Mediante una arquitectura resiliente se pueden agilizar los tiempos de atención de los nuevos requerimientos, la solución de problemas y mantener la disponibilidad de los servicios para los usuarios, contribuyentes y la ciudadanía en general.
- Una arquitectura resiliente permitirá a la institución adoptar una cultura DevSecOps que permita una comunicación más fluida entre las áreas involucradas en el proceso de despliegue de aplicaciones disminuyendo la cantidad de errores.

7. Recomendaciones

- Se recomienda que se designe un equipo multidisciplinario que permita la elaboración de un nuevo proceso de despliegue orientado a DevSecOps.
- Se recomienda realizar un análisis de compatibilidad de todos los componentes tecnológicos de los ambientes productivos y no productivos relacionados con la implementación de la plataforma de contenedores para validar la viabilidad de la instalación de la solución de contenedores en estos entornos.
- Es indispensable una intervención a los procedimientos relacionados al proceso de gestión tecnológica con la finalidad de realizar la actualización respectiva en función del nuevo proceso de despliegue que afecta al diseño, implementación y operación de servicios tecnológicos.
- Se recomienda la creación o actualización de estándares, guías y arquitecturas orientadas a la nueva arquitectura.
- Se recomienda que en base al Roadmap institucional de aplicaciones y a la criticidad de los servicios que presta la institución, se establezca el plan de adopción de la nueva arquitectura resiliente.
- Se recomienda la adquisición de una plataforma de contenedores *on premise* que permita la implementación de una arquitectura resiliente.

8. Bibliografía

- Apiumhub. (Julio de 2019). *El Patrón Retry: Ejemplos Y Recomendaciones*. Obtenido de El Patrón Retry: Ejemplos Y Recomendaciones: <https://apiumhub.com/es/tech-blog-barcelona/el-patron-retry/>
- Blancarte, O. (4 de Diciembre de 2018). *Circuit Breaker pattern*. Obtenido de <https://www.oscarblancarteblog.com/>: <https://www.oscarblancarteblog.com/2018/12/04/circuit-breaker-pattern/>
- Faye, F. (5 de Julio de 2022). *Resilient microservices with Quarkus and Kotlin - Design Patterns*. Obtenido de Dev.to: <https://dev.to/meikefelicia/resilient-microservices-with-quarkus-and-kotlin-design-patterns-i8o>
- Fedora. (2019). *Fedora Containers Lab*. Obtenido de Fedora Containers Lab: <https://fedoramx.fedorapeople.org/slides/Containers-Lab-Fedora.pdf>
- IBM. (2023). *¿Qué es Kubernetes?* Obtenido de IBM Topics: <https://www.ibm.com/es-es/topics/kubernetes>
- IBM. (2023). *Cómo las tecnologías de la Industria 4.0 están cambiando la fabricación*. Obtenido de IBM Topics: <https://www.ibm.com/es-es/topics/industry-4-0>
- Intel. (2023). *Microservicios y arquitectura de microservicios*. Obtenido de Computación nube: <https://www.intel.la/content/www/xl/es/cloud-computing/microservices.html#:~:text=La%20arquitectura%20de%20microservicios%20es,suelto%20de%20una%20interfaz%20est%C3%A1ndar>
- Microsoft. (28 de 03 de 2023). *Flujo de trabajo de desarrollo para aplicaciones de Docker*. Obtenido de Libro electrónico sobre la arquitectura de microservicios de .NET: <https://learn.microsoft.com/es-es/dotnet/architecture/microservices/docker-application-development-process/docker-app-development-workflow>
- Microsoft. (2023). *Guía de la arquitectura de aplicaciones*. Obtenido de Estilo de arquitectura de microservicios: <https://learn.microsoft.com/es-es/azure/architecture/guide/architecture-styles/microservices>
- NetApp. (2023). *¿Qué son los contenedores?* Obtenido de Soluciones DevOps: <https://www.netapp.com/es/devops-solutions/what-are-containers/>
- Rajeev Gupta [mydbsyn]. (2019). *HOW INTEGRATION PLATFORM IS DRIVING THE 4TH INDUSTRIAL REVOLUTION IN 2019 [Diagrama]*. Obtenido de mydbsyn: <https://www.mydbsync.com/blogs/how-integration-platform-is-driving-the-4th-industrial-revolution/>
- Red Hat. (2022). *Arquitecturas resilientes con modelos de automatización*. Quito.
- Red Hat. (2022). *El estado del opensource empresarial*. Obtenido de El estado del opensource empresarial: <https://www.redhat.com/rhdc/managed-files/rh-enterprise-open-source-report-f31123-202202-es.pdf>
- SAP. (2023). *¿Qué es la Industria 4.0?* Obtenido de Industry 4.0: <https://www.sap.com/latinamerica/products/scm/industry-4-0/what-is-industry-4-0.html>
- Tibco. (2023). *¿Qué es una arquitectura de microservicios? [Diagrama]*. Obtenido de Reference Center: <https://www.tibco.com/es/reference-center/what-is-a-microservices-architecture>
- Vinsguru. (7 de Noviembre de 2020). *Timeout Pattern – Microservice Design Patterns*. Obtenido de Vinsguru: <https://www.vinsguru.com/timeout-pattern/>

9. Glosario

- API: Interfaces de programación de aplicaciones utilizadas para la comunicación entre microservicios en una arquitectura de microservicios.
- Analíticas de la IA: Herramientas de análisis de datos impulsadas por inteligencia artificial que permiten a las empresas extraer información valiosa y tomar decisiones basadas en datos.
- Arquitectura de microservicios: Estilo arquitectónico en el que una aplicación se divide en pequeños servicios independientes que se comunican entre sí mediante interfaces definidas.
- Arquitectura monolítica: Arquitectura de software en la que todas las funciones de una aplicación están integradas en un solo bloque o módulo, lo que puede generar problemas de mantenimiento y actualización, así como una mayor dificultad para escalar la aplicación.
- Arquitectura resiliente: Diseño de software que se enfoca en la tolerancia a fallos y la capacidad de recuperación, mediante la implementación de estrategias de redundancia y escalabilidad.
- Arquitectura tradicional: Diseño de software basado en una estructura monolítica, donde los diferentes componentes y servicios se encuentran interconectados de manera jerárquica.
- Automatización de procesos de base de datos: Implementación de procesos automatizados para la gestión de la base de datos de una aplicación, lo que permite una mayor eficiencia y una reducción del riesgo de errores.
- Automatización del despliegue de aplicaciones: Implementación de un proceso automatizado para desplegar aplicaciones en diferentes entornos sin la necesidad de intervención manual.
- Big Data: Conjunto de datos masivos que requieren tecnología especializada para ser analizados y extraer información valiosa.
- CI/CD: Integración Continua/ Entrega Continua, un conjunto de prácticas de DevOps que busca reducir el tiempo entre la escritura del código y su entrega al usuario final, mediante la automatización de la construcción, pruebas y despliegue de software.
- Centro de datos on-premises: Centro de datos ubicado dentro de la misma organización o empresa, en lugar de estar alojado en la nube o en un proveedor externo.
- Certificación: Proceso mediante el cual se verifica que un producto, servicio o proceso cumple con determinados estándares de calidad y seguridad establecidos por una entidad autorizada.
- Change Failure Rate: Métrica que mide el porcentaje de despliegues de software que provocan una falla en producción.
- Ciberseguridad: Prácticas y tecnologías para proteger los sistemas informáticos y la información de posibles amenazas y ataques cibernéticos.
- Ciclo de vida de construcción: Proceso de construcción, prueba y despliegue de una aplicación, desde la creación
- Commit: se refiere al proceso de guardar los cambios realizados en el código fuente de una aplicación en un repositorio de control de versiones.
- Computación en la nube: Modelo de entrega de servicios de tecnología de la información a través de internet que permite a las empresas acceder a recursos informáticos escalables y flexibles.
- Contenedor: unidad de software que encapsula una aplicación y todas sus dependencias, permitiendo su portabilidad y ejecución en diferentes entornos.

- Contenerización: se refiere a la tecnología de contenedores, que permite ejecutar aplicaciones de forma aislada y portátil, asegurando la consistencia del entorno en el que se ejecutan.
- Continuidad del servicio: La capacidad de una organización para mantener la operación de los servicios y sistemas críticos en caso de fallos o interrupciones, asegurando que los usuarios puedan acceder a los servicios sin interrupciones.
- Cultura organizacional: conjunto de valores, creencias y prácticas compartidas por los miembros de una organización, que influyen en su comportamiento y desempeño. La cultura organizacional puede ser un factor determinante en el éxito o fracaso de una organización.
- Deployment Frequency: Métrica que mide la frecuencia con la que se realizan despliegues de software en producción.
- Despliegue: se refiere al proceso de implementar una aplicación o un cambio en una aplicación en un entorno de producción, lo que implica llevar la aplicación desde un entorno de prueba o desarrollo a uno de producción.
- DevSecOps: Una práctica que busca incorporar la seguridad en todo el ciclo de vida del software, desde el diseño hasta la operación, con el objetivo de minimizar riesgos y vulnerabilidades.
- DevOps: Prácticas y metodologías que buscan la integración entre los equipos de desarrollo y operaciones para lograr una entrega de software más rápida y confiable, mediante la automatización de procesos y la adopción de una cultura de colaboración y mejora continua.
- Escalabilidad: se refiere a la capacidad de una aplicación o servicio para crecer y adaptarse a un mayor número de usuarios o demandas sin perder calidad o rendimiento.
- Event-driven: describe un proceso en el cual una acción se desencadena por la ocurrencia de un evento específico.
- Flujos individuales de desarrollo: La arquitectura de microservicios permite que diferentes equipos trabajen de manera independiente en la construcción y desarrollo de cada microservicio, lo que se traduce en flujos individuales de desarrollo que pueden ser más ágiles y eficientes.
- Función como servicio (FaaS): Se refiere a una opción de implementación de microservicios en la nube que permite ejecutar código en respuesta a eventos o solicitudes específicas, sin necesidad de configurar y mantener servidores.
- Fábricas inteligentes: Fábricas equipadas con tecnología avanzada, como sensores, Internet de las cosas (IoT) y análisis de datos, para mejorar la eficiencia y la productividad.
- Homologación de ambientes: Se refiere al proceso de hacer que los ambientes de desarrollo, pruebas y producción sean consistentes y estén configurados de manera similar, lo que reduce la probabilidad de errores y facilita el proceso de despliegue.
- Imagen del sistema operativo: Un archivo que contiene una copia completa del sistema operativo, incluidos todos los archivos, bibliotecas y configuraciones necesarios para ejecutar una aplicación.
- Implementación continua: Una práctica de desarrollo de software que permite que las actualizaciones se realicen de manera continua y automática, en lugar de lanzamientos grandes y menos frecuentes. En una arquitectura de microservicios, los flujos individuales de desarrollo pueden implementarse de manera continua y rápida, lo que reduce el tiempo de espera para los usuarios finales.

- Innovación: Proceso de creación y aplicación de nuevas ideas, productos, servicios o procesos, que generan valor para la organización y sus clientes.
- Integración horizontal y vertical: Integración de diferentes sistemas y procesos en una empresa (horizontal) y la integración de diferentes etapas de la cadena de valor (vertical) para mejorar la eficiencia y la colaboración.
- Internet de las cosas industrial (IIoT): Conectividad de dispositivos industriales y sensores a internet para mejorar la eficiencia y la productividad de las fábricas.
- Kubernetes: plataforma de sistema distribuido de código libre para la automatización del despliegue, escalamiento y gestión de aplicaciones en contenedores.
- Lead Time for Change: Métrica que mide el tiempo que transcurre desde que se realiza un cambio en el código fuente hasta que se implementa en producción.
- Lean Startup: metodología utilizada en la creación de empresas y productos, que se enfoca en maximizar el aprendizaje validado a través de la experimentación temprana del mercado.
- Lenguajes de programación: Se refiere a los diferentes lenguajes que se pueden utilizar para construir componentes de una aplicación en una arquitectura de microservicios, lo que permite mayor flexibilidad en la elección de tecnologías.
- Mean Time To Restore: Métrica que mide el tiempo promedio utilizado para recuperarse de un problema causado por un despliegue de software.
- Mejora continua: proceso sistemático utilizado para mejorar la eficacia y eficiencia de los procesos y operaciones de una organización, mediante la identificación y eliminación de desperdicios y la implementación de mejores prácticas.
- Metodología ágil: conjunto de metodologías de desarrollo de software que se caracterizan por su enfoque iterativo e incremental, su capacidad de respuesta a los cambios y su orientación al cliente.
- Microservicios: se refiere a una arquitectura de software que descompone las aplicaciones en pequeños servicios independientes y escalables, lo que permite una mayor flexibilidad y modularidad en el desarrollo de aplicaciones.
- Nube Híbrida: Un modelo de infraestructura de TI que combina recursos de nube pública y privada, permitiendo una mayor flexibilidad y escalabilidad.
- Open source: término utilizado para referirse a software cuyo código fuente está disponible públicamente y puede ser utilizado, modificado y redistribuido libremente por cualquier persona.
- Orquestación de contenedores: Es el proceso de gestionar y coordinar los contenedores en un clúster de contenedores, incluyendo la asignación de recursos, la escalabilidad y el balanceo de carga.
- Pipeline: Un conjunto de herramientas y procesos que automatizan el flujo de trabajo del desarrollo de software, desde la integración de código, pruebas, despliegue y entrega.
- Plataforma como servicio (PaaS): Es un modelo de computación en la nube en el que un proveedor ofrece a los desarrolladores un conjunto de herramientas y servicios para crear, probar y desplegar aplicaciones en la nube.
- Plataforma de contenedores: Una plataforma que permite la creación, gestión y ejecución de contenedores, lo que facilita el despliegue y el manejo de aplicaciones.
- Portabilidad: La capacidad de una aplicación o servicio para ser transferido de un entorno a otro sin necesidad de realizar cambios significativos.

- Proceso de Gestión Tecnológica: conjunto de actividades y procesos que buscan gestionar de manera efectiva
- Producción en masa: Método de producción que se caracteriza por la fabricación de grandes cantidades de productos estandarizados en línea de montaje, lo que permite reducir los costos y aumentar la eficiencia.
- Realidad aumentada (AR): Tecnología que superpone información digital en el mundo real, lo que permite a los usuarios interactuar con objetos virtuales en un entorno físico.
- Red Hat Ansible Automation Platform: Una plataforma de automatización empresarial que permite la automatización de procesos y la gestión de infraestructuras, aplicaciones y redes.
- Red Hat OpenShift: plataforma principal de Kubernetes para las empresas, que permite diseñar, implementar y ejecutar las aplicaciones de manera uniforme en diferentes entornos.
- Robots autónomos: Robots programados para realizar tareas sin intervención humana, mediante el uso de sensores y algoritmos de inteligencia artificial.
- Seguridad Integrada: Una práctica de seguridad que busca incorporar la seguridad en todas las fases del ciclo de vida del software, desde el diseño hasta la operación, con el objetivo de minimizar riesgos y vulnerabilidades.
- Simulación/gemelos digitales: Réplicas digitales de procesos y sistemas físicos, que permiten a las empresas probar y simular diferentes escenarios antes de implementar cambios en el mundo real.
- Sobrecarga: El costo adicional de recursos de hardware y software necesarios para ejecutar una aplicación o sistema.
- Solución de gestión de contenedores: Una herramienta o plataforma que permite la administración de contenedores y la continuidad de los servicios desarrollados en contenedores para las aplicaciones de una organización.
- Team Topologies: Un marco de trabajo que describe diferentes formas en que los equipos pueden organizarse para lograr una entrega de software más efectiva, según las necesidades y características de cada organización.
- Teoría de las tecnologías: Campo de estudio que se enfoca en el impacto de las tecnologías en la sociedad y cómo éstas influyen en la cultura, la política, la economía y otros aspectos sociales.
- Transformación digital: El proceso de utilizar tecnologías digitales para transformar los procesos y modelos de negocio existentes y crear nuevos modelos de negocio para mantenerse al día con los cambios en el mercado.